CSCE 763 — Digital Image Processing
Distance Measurement & Mathematical Tools

Study Notes

Distance Measurement

A function $D(p, q)$ qualifies as a *distance metric* if it satisfies three properties for pixels p, q, and z:

(a) **Non-negativity:** $D(p, q) \geq 0$, and $D(p, q) = 0$ if and only if $p = q$

(b) **Symmetry:** $D(p, q) = D(q, p)$

(c) **Triangle inequality:** $D(p, z) \leq D(p, q) + D(q, z)$

Given pixel p at coordinates (x, y) and pixel q at coordinates (s, t), there are three standard distance measures.

Euclidean Distance

$$D_e(p, q) = \sqrt{(x - s)^2 + (y - t)^2} \tag{1}$$

This is the straight-line distance. Pixels at a given Euclidean distance from a center form a *circle*.

City-Block (D4) Distance

$$D_4(p, q) = |x - s| + |y - t| \tag{2}$$

Movement is restricted to horizontal and vertical directions only, like navigating a Manhattan street grid. Pixels at a given D4 distance from a center form a *diamond* shape. This distance corresponds to the N_4 neighborhood concept.

Chessboard (D8 / Chebyshev) Distance

$$D_8(p, q) = \max(|x - s|, |y - t|) \tag{3}$$

Movement is permitted in any of 8 directions, like a king in chess. Pixels at a given D8 distance from a center form a *square*. This corresponds to the N_8 neighborhood.

Sample Problem

Consider pixels p at the top-left corner and q at the bottom-right corner of a 6-row, 3-column region. The row difference is 5 and the column difference is 1.

$$D_4 = |5| + |1| = 6 \tag{4}$$

$$D_8 = \max(5, 1) = 5 \tag{5}$$

$$D_e = \sqrt{1 + 25} = \sqrt{26} \tag{6}$$

Key distinction: Distance is a direct measurement between two points. The *length of a path* is the sum of distances between consecutive pixels along a connected path, which is always $\geq$ the direct distance.

Array vs. Matrix Operations

This is a critical distinction in image processing.

Array Operations (Element-wise)

Each element in the result is computed from the corresponding elements in the same position. For multiplication:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} \\ a_{21}b_{21} & a_{22}b_{22} \end{bmatrix} \tag{7}$$

Position (i, j) in the output is simply the product of position (i, j) from each input. This is what image arithmetic (averaging, subtraction, multiplication) uses — pixel-by-pixel operations.

Matrix Operations (Standard Linear Algebra)

Uses dot products of rows and columns:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix} \tag{8}$$

Matrix multiplication is used for *geometric transformations* (affine transforms like rotation, scaling, translation, shear) and vector/matrix operations in color image processing.

Bottom line: Most pixel-level image processing uses *array* (element-wise) operations. *Matrix* multiplication is reserved for coordinate transformations and linear algebra tasks.

Linear vs. Nonlinear Operations

An operator H is *linear* if it satisfies the superposition property:

$$H\big[a_i f_i(x, y) + a_j f_j(x, y)\big] = a_i H\big[f_i(x, y)\big] + a_j H\big[f_j(x, y)\big] \tag{9}$$

This combines two properties:

Additivity: $H[f_1 + f_2] = H[f_1] + H[f_2]$. The response to a sum of inputs equals the sum of individual responses.

Homogeneity: $H[a \cdot f] = a \cdot H[f]$. Scaling the input scales the output by the same factor.

Examples of linear operations: image averaging, spatial convolution with a fixed kernel, Fourier transforms.

Examples of nonlinear operations: median filtering, thresholding, taking the max or min of pixel values. These break the superposition property because, for example, the median of a sum is not the sum of the medians.

Linearity matters because linear operations are mathematically tractable — they can be analyzed in the frequency domain, their behavior can be predicted, and complex operations can be decomposed into simpler ones.

Applications of Image Arithmetic

Image Averaging — Noise Reduction

A noisy image is modeled as $g(x, y) = f(x, y) + \eta(x, y)$, where f is the true image and η is random noise. If K noisy images of the same scene are captured and averaged:

$$\bar{g}(x, y) = \frac{1}{K} \sum_{i=1}^{K} g_i(x, y) \tag{10}$$

Then the expected value converges to the true image and the noise variance is reduced:

$$E\{\bar{g}(x, y)\} = f(x, y) \tag{11}$$

$$\sigma_{\bar{g}}^2 = \frac{1}{K} \sigma_{\eta}^2 \tag{12}$$

Averaging K images reduces noise by a factor of K. The assumption is that the noise is *uncorrelated* between images and has *zero mean*.

Image Subtraction — Enhancing Differences

$$g(x, y) = f_1(x, y) - f_2(x, y) \tag{13}$$

There are two key applications. The first is **detecting changes**: subtracting two similar images highlights what changed between them. The second is **Digital Subtraction Angiography (DSA)**: a "mask" X-ray image is taken before injecting contrast dye, and a "live" image is taken after injection. Subtracting them cancels out all static anatomy (bones, tissue) and reveals only the blood vessels filled with dye.

Image Multiplication — ROI Masking

$$g(x, y) = f(x, y) \cdot h(x, y) \tag{14}$$

A binary mask h is created with 1s where the region of interest should be preserved and 0s elsewhere. Multiplying the mask with the image isolates the desired region. For example, a dental X-ray can be multiplied with a rectangular mask to isolate only the teeth with fillings.

Image Division — Shading Correction

$$g(x, y) = f(x, y) \,/\, h(x, y) \tag{15}$$

If an image has non-uniform illumination (common in microscopy), the shading pattern $h(x, y)$ can be estimated and divided out. This corrects gradual brightness falloff across the image.

Important Notes on Arithmetic Operations

Images used in averaging and subtraction **must be registered** (spatially aligned). Misalignment will produce artifacts instead of meaningful results.

Output images should be **normalized to** $[0, 255]$ using:

$$f_m = f - \min(f) \tag{16}$$

$$f_s = K \left[\frac{f_m}{\max(f_m)} \right], \quad K = 255 \tag{17}$$